Université Tahri Mohamed de Béchar
Faculté des sciences exactes
Département des Mathématiques et Informatique
2ème Année Ingénieur Informatique

Matière:

Algorithmiques et complexité

Cours n° 04

Enseignant : Benyamina Ahmed

Matière: Algorithmiques et complexité

Programme de la matière :

I- Rappels

II- Notion de complexité algorithmique

III- Algorithmes de tri

1. Présentation
2. Tri à Bulles (Bubble-Sort)
3. **Tri par sélection (Selection-sort)**
4. **Tri par insertion (Insertion-sort)**
5. **Tri fusion (Merge-sort)**
6. **Tri rapide (quick-sort)**
7. **Conclusion**

IV- Les arbres

V- Les graphes

Algorithmes de tri

Rappel: Tri à Bulles (Bubble sort)

La mise en œuvre du tri à bulles en Python

```
def bubbleSort(array):
  n = len(array)
    for i in range(n-1):
      for j in range(0, n-i-1):
        if array[j] > array[j+1]:
          array[j], array[j+1] = array[j+1], array[j]
```

Règle:

La complexité de tout algorithme de tri dépend du nombre de comparaison .

Pire des cas, Moyenne des cas et meilleur des cas: $O(n^2)$

Algorithmes de tri

2- Tri par sélection (Selection sort) - Présentation

Le tri par sélection est conceptuellement l'algorithme de tri le plus simple.

Cet algorithme trouve d'abord le plus petit élément du tableau et l'échange avec l'élément en première position, puis trouve le deuxième plus petit élément et l'échange avec l'élément en deuxième position, et continue ainsi jusqu'à ce que tout le tableau soit trié.

Algorithmes de tri

2- Tri par sélection (Selection sort) - Présentation

Voici les étapes à effectuées dans ce tri:

1. Commencez par une liste d'éléments non triés.
2. Définir le premier élément comme valeur minimale.
3. Parcourir la partie non triée de la liste, en commençant par le deuxième élément.
4. Comparer chaque élément avec la valeur minimale actuelle.
5. Si un élément plus petit est trouvé, mettre à jour la valeur minimale avec l'élément nouvellement trouvé.
6. Après avoir parcouru la partie non triée, la valeur minimale sera le plus petit élément de la partie non triée.
7. Echanger la valeur minimale avec le premier élément de la partie non triée. Cela place la valeur minimale au début de la partie triée de la liste.

Algorithmes de tri

2- Tri par sélection (Selection sort) - Présentation

8. Déplacer la limite de la partie triée d'une position vers la droite.

9. Répéter les étapes 3 à 8 jusqu'à ce que toute la liste soit triée.
   Cela signifie que la partie triée s'agrandira progressivement de gauche à droite et que la partie non triée diminuera en conséquence.

Algorithmes de tri

2- Tri par sélection (Selection sort) - Présentation

10. Une fois l'algorithme terminé, la liste sera triée par ordre croissant..

Algorithmes de tri

2- Tri par sélection (Selection sort) - Présentation

La mise en œuvre du tri par sélection en Python est :

```
def selection_sort(arr):
  n = len(arr)
  for i in range(n-1):
      min_index = i
      for j in range(i+1, n):
          if arr[j]< arr[min_index]:
              min_index = j
       arr[i], arr[min_index] = arr[min_index], arr[i]
```

Algorithmes de tri

2- Tri par sélection (Selection sort) - Complexité

- Le premier élément est comparé aux n-1 éléments restants dans la passe 1.
- Ensuite, n-2 éléments sont pris dans la passe 2, ce processus est répété jusqu'à ce que le dernier élément soit rencontré.
- L'expression mathématique de ces itérations sera égale à (n-1)+(n-2)+....+(n-(n-1)),.
- Ainsi l'expression devient **n*(n-1)/2.**
- Ainsi , le nombre de comparaisons est proportionnel à (n^2).
- La complexité temporelle du tri par sélection est $O(n^2)$.

Pire des cas, Moyenne des cas et meilleur des cas: $O(n^2)$

Algorithmes de tri

2- Tri par insertion (Insertion sort) - Présentation

Le tri par insertion est un algorithme de tri qui commence par considérer le premier élément du tableau comme une séquence triée. Ensuite, il insère progressivement chaque élément non trié dans la séquence triée de manière à ce que la séquence triée s'étende au fur et à mesure

Algorithmes de tri

2- Tri par insertion (Insertion sort) - Présentation

1. Commencez par le deuxième élément (index 1) et considérez-le comme la clé.
2. Comparez la clé avec le ou les éléments à sa gauche dans la partie triée du tableau.
3. Si la clé est plus petite, déplacez les éléments les plus grands d'une position vers la droite pour laisser de la place à la clé.
4. Répétez l'étape 3 jusqu'à ce que vous trouviez la position correcte pour la clé ou atteigniez le début du tableau.
5. Insérez la clé dans la bonne position.
6. Passez à l'élément suivant et répétez les étapes 2 à 5 jusqu'à ce que tous les éléments soient traités.

Algorithmes de tri

2- Tri par insertion (Insertion sort) - Présentation

La mise en œuvre du tri par Insertion en Python est :

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        for j in range(i - 1, -1, -1):
       # Boucle "for" avec un pas négatif
            if arr[j] > key:
                arr[j + 1] = arr[j]
            else:
                break
        arr[j + 1] = key
```

Algorithmes de tri

2- Tri par insertion (Insertion sort) - Complexité

- Dans le tri par insertion, nous insérons les éléments i avant ou après et nous commençons la comparaison à partir du premier élément.
- Le dernier élément nécessite n - 1 comparaisons.
- le pire comportement du tri par insertion où tous les éléments sont dans l'ordre inverse.
- Si nous calculons le cas moyen de l'algorithme ci-dessus, alors ce sera $O(n^2)$.
- La technique du tri par insertion est très efficace si le nombre d'éléments à trier est très inférieur.
- Si les éléments sont déjà triés alors O(n)

Pire des cas et Moyenne des cas : $O(n^2)$

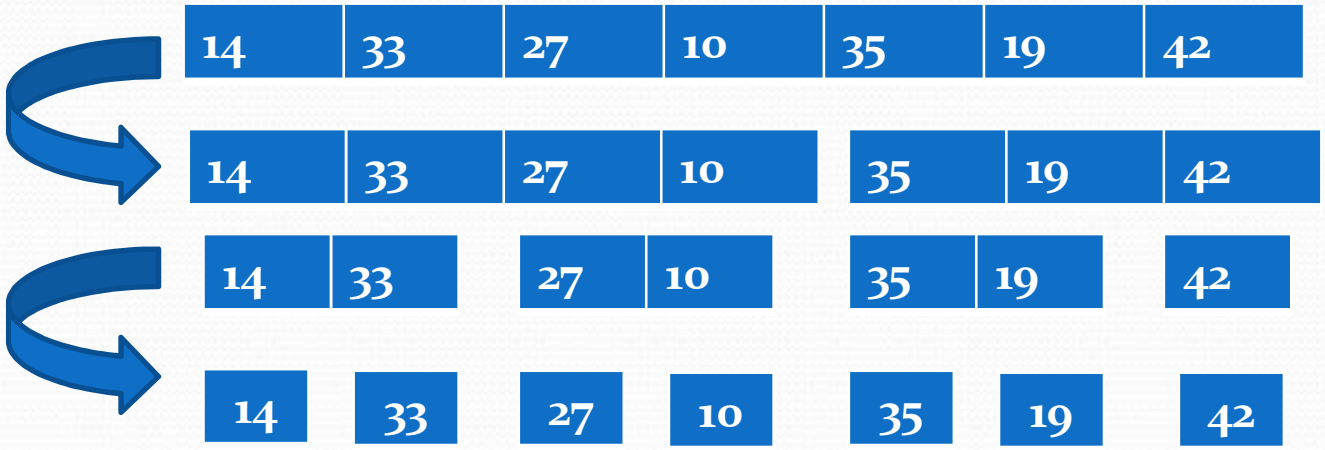
Meilleur des cas: O(n)

Algorithmes de tri

2- Tri par fusion (Merge sort) - Présentation

Le tri par fusion est une technique de tri basée sur la technique de diviser pour mieux régner. Le tri par fusion divise d'abord le tableau en moitiés égales, puis les fusionne de manière triée.

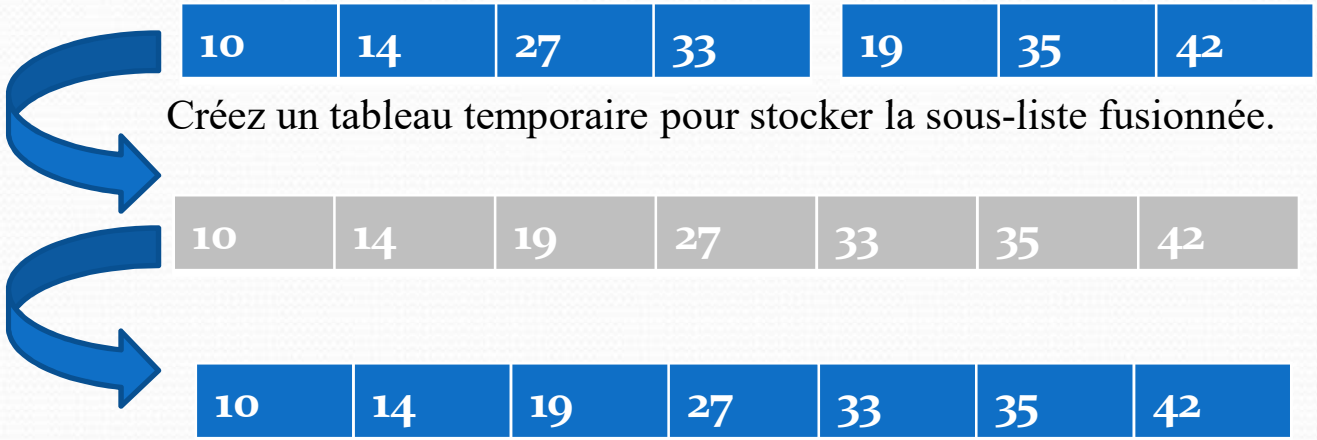
1. Division : La liste non triée est divisée récursivement en sous-listes plus petites jusqu'à ce que chaque sous-liste ne contienne qu'un seul élément. - Si la liste contient zéro ou un élément, elle est déjà considérée comme triée et ne nécessite aucun traitement supplémentaire. - Sinon, la liste est divisée en deux sous-listes en trouvant l'index du milieu : Définissez l'index du milieu comme mid = (gauche + droite) / 2, où left est l'index de début de la sous-liste et right est l'index de fin. Divisez récursivement la sous-liste de gauche de la gauche au milieu. Divisez récursivement la sous-liste de droite du milieu + 1 vers la droite.



Algorithmes de tri

2- Tri par fusion (Merge sort) - Présentation

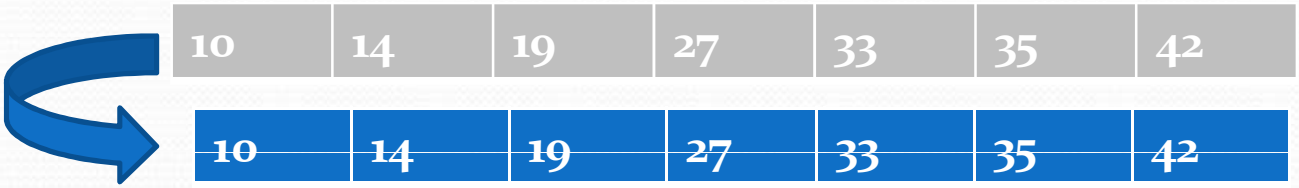
2. Fusion : Les sous-listes triées sont fusionnées pour créer une seule liste triée. Créez un tableau temporaire pour stocker la sous-liste fusionnée. Initialisez trois pointeurs : i pour la sous-liste de gauche, en commençant à l'index gauche. j pour la sous-liste de droite, commençant à l'index mid + 1. k pour la sous-liste fusionnée, en commençant à l'index gauche. Comparez les éléments aux indices i et j des deux sous-listes. Si l'élément à l'index i est plus petit (ou égal), copiez-le dans la sous-liste fusionnée et incrémentez i et k. Si l'élément à l'index j est plus petit, copiez-le dans la sous-liste fusionnée et incrémentez j et k.



Algorithmes de tri

2- Tri par fusion (Merge sort) - Présentation

2. Fusion : Répétez le processus de comparaison et de copie jusqu'à ce qu'une sous-liste soit épuisée. S'il reste des éléments dans la sous-liste de gauche, copiez-les dans la sous-liste fusionnée. S'il reste des éléments dans la sous-liste de droite, copiez-les dans la sous-liste fusionnée. Remplacez les éléments de la liste d'origine des index de gauche à droite par les éléments triés de la sous-liste fusionnée.
3. Répéter : répétez de manière récursive les étapes de division et de fusion pour chaque sous-liste divisée jusqu'à ce que la liste entière soit triée.



Voici en cette vidéo toutes les étapes citées précédemment.

Algorithmes de tri

2- Tri par Fusion (Merge sort) - Présentation

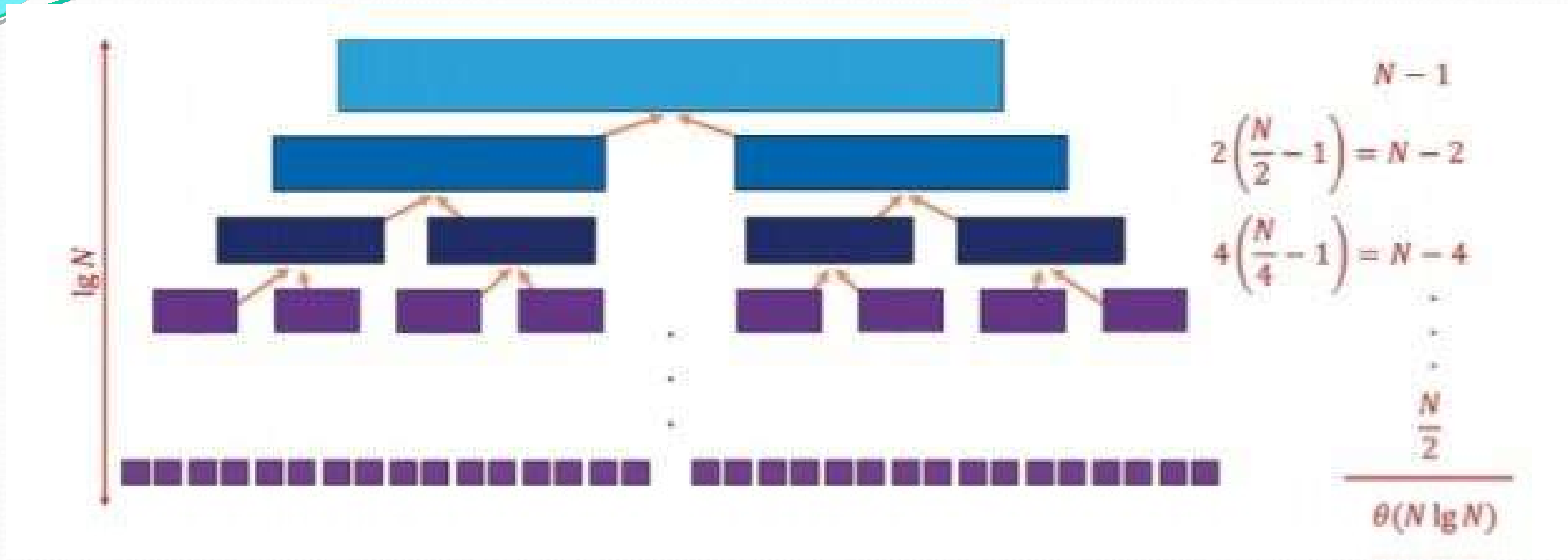
La mise en œuvre du tri par fusion en Python est :

```
def merge_sort(array, aux_array, start, end):
    If start ==end:
        return
    Else:
        mid = int((star + end )  / 2 )
        merge_sort(array, aux_array, start, mid)
        merge_sort(array, aux_array, mid+1, end)
        merge_both_parts(array, aux_array, start,
        mid+1, end)
```

```
def merge_both_parts(array, aux_array, left, right, end)
    start = left
    mid = right – 1
    items_count = end – left +1
    I = 0
    while (left <= mid and right <= end ):
        If array[left] < array[right]):
            aux_array[i] = array[left]
            i += 1
            left += 1
        else:
            aux_array[i] = array[right]
            i += 1
            right += 1
    while left <= mid:
            aux_array[i] = array[left]
            i += 1
            left += 1
    while right<= end:
            aux_array[i] = array[lright]
            i += 1
            right+= 1
        i= 0
    while i< item_count:
        array[start + 1] = aux_array[right]
            i += 1
```

Algorithmes de tri

2- Tri par fusion (Mergesort) - Complexité



La complexité temporelle du tri par fusion est :

O(n.Log(n))

car le tri par fusion divise toujours le tableau en deux moitiés et prend un temps linéaire pour fusionner deux moitiés.

Algorithmes de tri

2- Tri rapide (Quick sort) - Présentation

Le tri rapide trie en utilisant une stratégie diviser pour mieux régner pour diviser une liste en deux sous-listes.

Ce tri a été inventé par **Charle Richard Hoare** en 1960

Les étapes sont les suivantes :

1. Choisissez un élément, appelé pivot, dans la liste.
2. Réorganisez la liste de sorte que tous les éléments inférieurs au pivot viennent avant le pivot et que tous les éléments supérieurs au pivot viennent après (des valeurs égales peuvent aller dans les deux sens). Après ce cloisonnement, le pivot est dans sa position définitive. C'est ce qu'on appelle l'opération de partition.
3. Triez récursivement la sous-liste des éléments inférieurs et la sous-liste des éléments supérieurs.

Algorithmes de tri

2- Tri rapide (Quick sort) - Présentation

Voici les étapes à effectuées dans ce tri:

Algorithmes de tri

2- Tri par (Quicksort) - Présentation

Amélioration du tri par rapide en choisissant le pivot au centre de la liste :

```
def partition(array, low, high):
    pivot = array[(low + high) // 2]
    i = low-1
    j = high+1
    while True:
        while true:
            i+=1
            if not (array[i] < pivot):
                break
        while true:
            j-=1
            if (array[j] > pivot):
                break
        if i >= j:
            return j
        array[i], array[j] = array[j], array[i]
def quickSort(array, low, high):
    if low < high:
        pivotIndex = partition(array, low, high)
        quickSort(array, low, pivotIndex)
        quickSort(array, pivotIndex + 1, high)
```

Algorithmes de tri

2- Tri par rapide(Quicksort) – Complexité : pire des cas

La complexité en pire des cas: Cela se produit principalement lorsque le pivot choisi à chaque étape divise le tableau de manière **déséquilibrée**, c'est-à-dire lorsque le pivot ne divise pas le tableau en parties relativement égales.
Dans ce cas, le pivot choisi au début du tri rapide sera soit le plus petit, soit le plus grand élément du tableau. Lorsque le pivot est le plus petit ou le plus grand élément, le tri rapide ne divise pas efficacement le tableau en deux parties de taille similaire, ce qui entraîne de nombreuses itérations et un grand nombre de ***récursions***. À chaque étape, le pivot n'élimine qu'un élément du tableau, de sorte que l'algorithme doit répéter ce processus n - 1 fois (n étant la taille du tableau).
Cela signifie que le tri rapide effectuera n - 1 itérations, et à chaque itération, il devra parcourir n - 1 éléments dans la première partie du tableau, puis n - 2 éléments, puis n - 3 éléments, et ainsi de suite. La somme totale des itérations sera :
$(n - 1) + (n - 2) + (n - 3) + ... + 1$
La somme de cette série est égale à $(n * (n - 1)) / 2$, ce qui conduit à une complexité en temps en $O(n^2)$ dans le pire cas.
Donc, la complexité est en temps de $O(n^2)$. Pour éviter le pire cas, il est important de choisir un pivot judicieux (par exemple, en utilisant la médiane des trois éléments du milieu) ou en utilisant des techniques d'optimisation pour améliorer la répartition des éléments autour du pivot. **Pire des cas : $O(n^2)$**

La complexité en meilleur des cas du tri rapide (quicksort) se produit lorsque le pivot est toujours choisi de manière à diviser le tableau de manière équilibrée, c'est-à-dire lorsque le pivot divise le tableau en deux parties de taille approximativement égale.

1. Le pivot est choisi de manière à diviser le tableau en deux parties presque égales.
2. Chaque élément dans le tableau est comparé une seule fois avec le pivot et est placé dans l'une des deux partitions.
3. Les deux partitions sont triées récursivement.

L'étape 3 est répétée pour les partitions jusqu'à ce que le tableau entier soit trié.
Donc la complexité en meilleur des cas du tri rapide est en temps de O(n log n).

Moyenne des cas et meilleur des cas : O(n log n)

Algorithmes de tri

Conclusion

Algorithmes	Meilleure des cas	Moyenne des cas	Pire des cas
Tri à Bulles	$O(n^2)$	$O(n^2)$	$O(n^2)$
Tri par insertion	**$O(n^2)$**	**$O(n^2)$**	**$O(n)$**
Tri par sélection	**$O(n^2)$**	**$O(n^2)$**	**$O(n^2)$**
Tri par fusion	**$O(n.\log(n))$**	**$O(n.\log(n))$**	**$O(n.\log(n))$**
Tri rapide	**$O(n.\log(n))$**	**$O(n.\log(n))$**	**$O(n^2)$**